METROPOLITAN UNIVERSITY, BANGLADESH

Department of Computer Science and Engineering

COMPILER CONSTRUCTION LABORATORY

PROJECT REPORT

Design and Implementation of a Mini Programming Language Compiler using Flex and Bison

Complete Front-End with Abstract Syntax Tree,
Semantic Analysis, Scoped Symbol Table, and Three-Address Code

Submitted by	Mark Pranto Sarkar - (231-115-270)
Project contributor	Tanmoy Das - (231-115-257)
Course	Compiler Construction Laboratory
Document edition	Final Submission - Version 2.0

Academic Year 2026

Executive Summary

This project implements the complete compiler front-end and intermediate-code pipeline required by the course manual. The compiler recognizes the fixed instructor-defined language, constructs an Abstract Syntax Tree (AST), maintains a nested-scope symbol table, applies semantic and type rules, and generates validated Three-Address Code (TAC). The final submission includes formal Flex and Bison specifications, a portable reviewed C11 implementation, automated regression tests, expected outputs, screenshots, build automation, and this report.

Verification gate	Final status
Strict C11 compilation	Passed with -Wall -Wextra -Wpedantic and no warnings
Regression suite	29/29 tests passed
Valid programs	13/13 accepted through TAC
Invalid programs	16/16 rejected with non-zero status
Sanitizer sweep	AddressSanitizer and UndefinedBehaviorSanitizer passed
Required phases	Lexer, parser, AST, symbol table, semantics, and TAC present

Rubric Compliance Matrix

Required item	Implementation evidence
Lexical Analysis	src/lexer/lexer.c, lexer.h, formal lexer.l, lexical error tests
Syntax Analysis	src/parser/parser.c, parser.h, formal parser.y, recovery tests
Abstract Syntax Tree	src/ast/ast.c and ast.h; two readable print formats
Symbol Table	src/symbol_table; nested scope and declaration metadata
Semantic Analysis	src/semantic; declaration, type, expression, and condition rules
Three-Address Code	src/tac; temporaries, labels, jumps, and validator
Project Report	docs/Project_Report.pdf plus Markdown source

Contents

The chapter numbering follows Section 12 of the authoritative project manual.

5. Introduction

Compiler construction combines several formally distinct phases into a single dependable pipeline. Source characters first become tokens; tokens are checked against a grammar; successful syntax becomes a structured tree; declarations and expressions are validated against language rules; and the verified tree is translated into an intermediate representation. This project integrates those responsibilities into one modular C11 program.

The implementation follows the language and deliverables defined in the Compiler Construction Lab Project Manual. It deliberately focuses on the required front-end and TAC stages rather than machine-code generation. A valid source program proceeds through every phase and produces an AST, a symbol table, and TAC. An invalid program is rejected with a non-zero process status and a categorized diagnostic containing an error code and source location whenever available.

The repository preserves the educational character of the project. Each compiler phase has its own directory and public interface. Data structures use descriptive names, the code avoids unnecessary abstraction, and test programs isolate both successful behavior and expected failures. Formal lexer.l and parser.y files document the Flex/Bison formulation required by the manual, while the default build uses reviewed portable C sources so the submission remains reproducibly buildable on systems without generator tools.

5.1 Project Context

The course project replaces the laboratory final examination and is evaluated through implementation, documentation, demonstration, presentation, and individual viva. Therefore, correctness alone is insufficient: the submission must also be readable, demonstrable, and explainable by every team member. The final organization and documentation were designed with that evaluation model in mind.

5.2 Scope Boundary

The required endpoint is Three-Address Code. Assembly generation, machine code, linking, register allocation, and hardware-specific backend work are intentionally excluded. Optional constant folding is included only after the complete mandatory pipeline and does not alter the fixed language grammar.

6. Objectives

1. Implement all six mandatory compiler modules as one coherent pipeline.

2. Recognize exactly the fixed language specified by the instructor.

3. Construct a readable AST that removes punctuation and preserves program meaning.

4. Implement nested lexical scopes and record identifier metadata.

5. Detect undeclared use, same-scope redeclaration, scope violations, type mismatches, invalid assignments, and invalid expressions.

6. Generate TAC for expressions, assignments, output, conditionals, and loops.

7. Produce clear lexical, syntax, and semantic diagnostics without hanging or failing silently.

8. Provide professional repository organization, build instructions, examples, expected outputs, screenshots, and a formal project report.

9. Demonstrate correctness through automated valid and invalid test programs.

10. Keep the code readable enough for presentation and individual viva explanation.

6.1 Success Criteria

Criterion	Measurable evidence
Buildability	One make command creates the compiler from source
Correct success path	Non-trivial valid program reaches AST, symbol table, and TAC
Correct rejection path	Lexical, syntax, and each required semantic rule have invalid tests
Phase separation	Dedicated module, header, and diagnostics for each phase
Documentation	README, formal language specification, report, screenshots, expected outputs
Repeatability	Automated suite and deterministic process exit statuses

7. Language Specification

7.1 Data Types

Type	Meaning	Representative literal
int	Signed integer value	42
float	Floating-point value	3.14
bool	Logical truth value	true / false

7.2 Lexical Elements

- Keywords: int, float, bool, if, else, while, print, true, false.
- Identifiers: a letter or underscore followed by letters, digits, or underscores.
- Arithmetic operators: +, -, *, /, %.
- Relational and equality operators: <, >, <=, >=, ==, !=.
- Logical operators: &&, ||, !.
- Assignment and delimiters: =, {, }, (,), ;.
- Comments: // to end of line and /* ... */ block comments.
- Whitespace and comments are discarded before parsing.

7.3 Complete Context-Free Grammar

```
program         ::= { statement } EOF
statement       ::= declaration | assignment | if_statement
                  | while_statement | print_statement | block | ";"
declaration     ::= type identifier [ "=" expression ] ";"
assignment      ::= identifier "=" expression ";"
if_statement    ::= "if" "(" expression ")" block [ "else" block ]
while_statement ::= "while" "(" expression ")" block
print_statement ::= "print" expression ";"
block           ::= "{" { statement } "}"
type            ::= "int" | "float" | "bool"
expression      ::= logical_or
logical_or      ::= logical_and { "||" logical_and }
logical_and     ::= equality { "&&" equality }
equality        ::= relational { ("==" | "!=") relational }
relational      ::= additive { ("<" | ">" | "<=" | ">=") additive }
additive        ::= multiplicative { ("+" | "-") multiplicative }
multiplicative  ::= unary { ("*" | "/" | "%") unary }
unary           ::= ("!" | "-") unary | primary
primary         ::= identifier | literal | "(" expression ")"
literal         ::= integer_literal | float_literal | boolean_literal
```

7.4 Precedence and Associativity

Priority	Operators / form	Associativity
1 (highest)	parenthesized primary	grouped
2	unary ! and unary -	right
3	* / %	left
4	+ -	left
5	< > <= >=	left

Priority	Operators / form	Associativity
6	== !=	left
7	&&	left
8 (lowest)	||	left

7.5 Scope and Type Rules

The program begins in global scope. Every block creates a nested lexical scope. A declaration is visible in its scope and descendant scopes until that scope ends. Redeclaration in the same scope is illegal; an identifier declared in an inner scope is inaccessible after the block exits.

Arithmetic operators require numeric operands. Modulo requires two integers. Relational operators require numeric operands and produce bool. Logical operators require boolean operands. Equality operands must be compatible. If and while conditions must be boolean. Assignment must respect the target declaration type; the implementation permits safe widening from int to float and rejects incompatible conversions.

8. Compiler Architecture

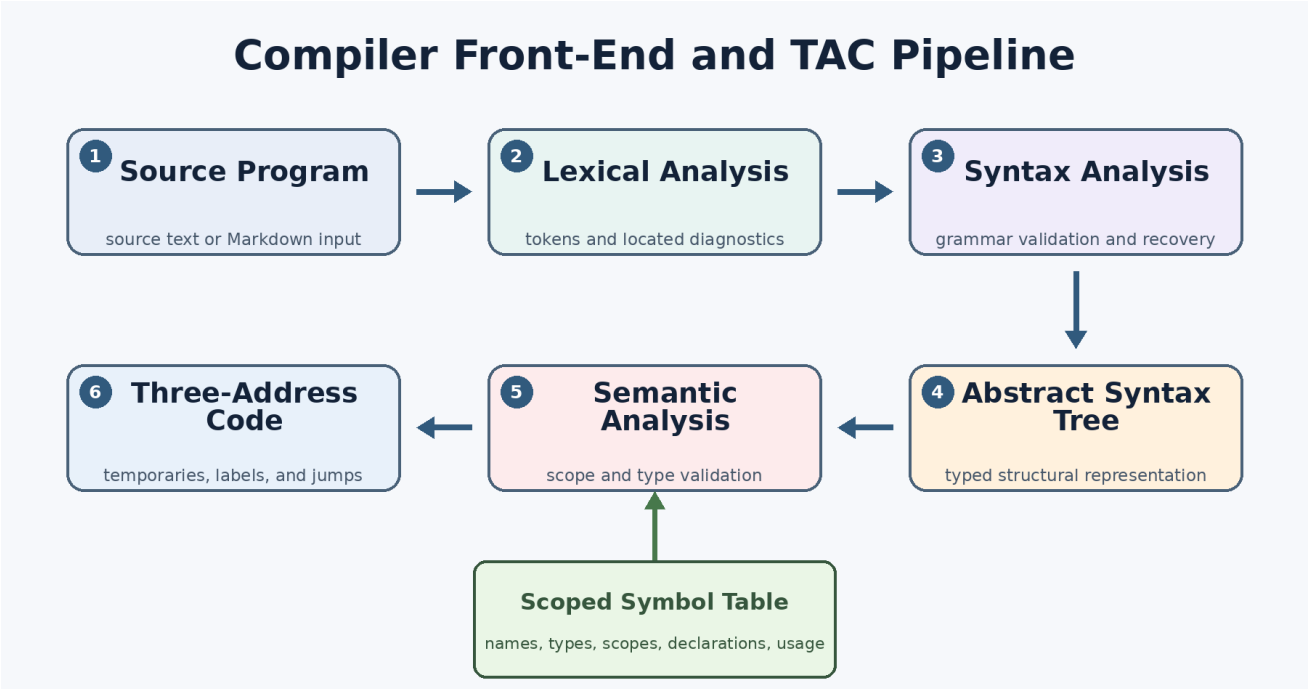

Figure 1. End-to-end compiler pipeline and symbol-table interaction.

The implementation is separated into phase-owned modules. The AST is the principal contract between parsing, semantics, and TAC generation. Semantic analysis annotates expression nodes with resolved types. TAC generation runs only after lexical, syntax, and semantic success, preventing invalid trees from producing misleading intermediate code.

Module	Primary responsibility
src/lexer	source scanning, token construction, comment handling, lexical diagnostics
src/parser	CFG enforcement, precedence, recovery, AST construction
src/ast	node ownership, type annotations, printing, counting, destruction
src/symbol_table	declaration storage, scoped lookup, usage tracking
src/semantic	declaration/type rules, expression validation, optional folding
src/tac	temporary/label allocation, code emission, TAC validation
src/main.c	CLI, source loading, phase orchestration, timing, cleanup

8.1 Phase Data Flow

The driver loads the source and passes it through lexical analysis, syntax analysis and AST construction, scoped semantic validation, and TAC generation. Each phase reports located diagnostics; TAC is emitted only for a semantically valid AST, checked for label consistency, and followed by deterministic cleanup.

9. Lexer Design

9.1 Recognition Rules

The scanner uses longest meaningful tokens first. Multi-character operators such as <=, >=, ==, !=, &&, and || are recognized before their one-character prefixes. Keyword recognition occurs before generic identifier recognition in the formal Flex file. Numeric scanning distinguishes integer and floating-point literals.

Category	Regular expression / rule	Action
Identifier	[A-Za-z_][A-Za-z0-9_]*	keyword or IDENTIFIER
Integer	[0-9]+	INT_LITERAL
Float	[0-9]+\.[0-9]+	FLOAT_LITERAL
Whitespace	spaces, tabs, newlines	discard; update location
Line comment	//[^\n]*	discard
Block comment	/* ... */	discard or report EOF error
Unmatched character	any remaining character	lexical diagnostic + INVALID

9.2 Diagnostics and Locations

The scanner records line and column positions. Invalid characters produce ERR_LEXICAL_INVALID_CHAR; an unfinished block comment produces ERR_LEXICAL_UNTERMINATED_COMMENT. The parser receives a dedicated invalid token so error handling is explicit rather than accidental.

```
[ERR_LEXICAL_INVALID_CHAR] Invalid character '@' at line 5, column 1:
Invalid character in source code
```

9.3 Formal Flex File and Portable Scanner

src/lexer/lexer.l is the formal Flex specification and contains the regular-expression rules expected for course review. The default executable uses lexer.c, a reviewed equivalent scanner with the same token model and additional column tracking. make flex-bison-check regenerates formal sources into build/generated when Flex and Bison are installed, without overwriting the tested portable implementation.

9.4 Resource Management

Token text and captured comments are dynamically allocated only when needed. lexer_destroy releases scanner-owned memory on every parser exit path, including failed parses. This behavior was verified under AddressSanitizer leak detection.

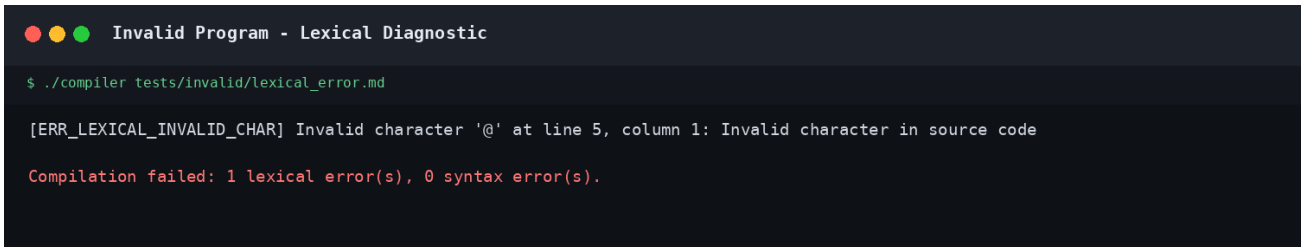

Figure 2. Actual lexical-error execution captured from the final compiler.

10. Parser Design

10.1 Grammar Implementation

The parser recognizes declarations, assignments, blocks, if, if-else, while, print, and empty statements. Expression parsing is divided into one function or nonterminal per precedence level. This structure preserves precedence without relying on ad hoc post-processing.

src/parser/parser.y provides the formal Bison grammar, semantic-value declarations, source locations, AST construction actions, and an error SEMICOLON recovery production. src/parser/parser.c is the submission executable's directly compiled, reviewed recursive-descent implementation of the same grammar.

10.2 AST Construction Actions

Successful productions allocate AST nodes immediately. Declaration nodes own an identifier child and, when present, an initializer child. Binary and unary productions store the operator in the node text. Blocks own their contained statements. Parentheses and semicolons are intentionally omitted because they no longer carry semantic meaning after parsing.

10.3 Error Recovery and Progress Guarantees

Syntax diagnostics include standardized codes for unexpected tokens and missing punctuation. Recovery skips input until a statement boundary such as a semicolon or closing brace. The statement-list loop also checks that parsing consumed input; if no progress occurred, it advances safely. This prevents malformed input such as int ; from causing an infinite loop.

Lexical and syntax error counts are kept separately. A lexical failure therefore cannot be mislabeled as a parser error in the final compilation summary.

Parser concern	Implemented response
Missing statement terminator	specific missing-semicolon diagnostic and synchronization
Unexpected token	located syntax diagnostic
Malformed declaration	expected-identifier diagnostic plus progress guard
Multiple errors	panic-mode synchronization at statement boundaries
Lexical INVALID token	lexical count retained separately from syntax count

11. Abstract Syntax Tree

11.1 Node Representation

Field	Purpose
NodeKind kind	program, statement, expression, literal, or identifier category
char *text	operator, literal, declared type, or descriptive label
int line	source location for display and diagnostics
DataType data_type	type assigned during construction or semantic analysis
children	dynamic owned list supporting variable-arity nodes
comment	optional associated source comment

Supported node kinds are Program, Block, Declaration, Assignment, If, While, Print, BinaryOp, UnaryOp, Literal, and Identifier. This is intentionally compact: every node corresponds to a meaningful language construct.

11.2 Tree Output and Ownership

The compiler prints both a conventional indented AST and a visual branch tree. Type annotations inserted by semantic analysis are included, making it possible to inspect structure and inferred expression types together. ast_count_nodes supports statistics, while recursive ast_free enforces clear ownership and deterministic cleanup.

11.3 Example

```
Assignment (=)
|- Identifier (y)
`- BinaryOp (+) : int
   |- Identifier (y) : int
   `- Identifier (x) : int
```

The assignment above preserves the target and expression structure while omitting the concrete semicolon and grammar-only nonterminals.

Valid Program - Full Compiler Pipeline

```
$ ./compiler tests/valid/full_demo.md

Symbol Table
------------
=== Detailed Symbol Table (Scope Level: 0) ===
All Symbols:
  - passed : bool (declared at line 3, scope 0) [ACTIVE]
  - i : int (declared at line 2, scope 0) [ACTIVE]
  - total : int (declared at line 1, scope 0) [ACTIVE]
================================================

Three Address Code
------------------
total = 0
i = 5
passed = true
L1:
t1 = i > 0
ifFalse t1 goto L2
t2 = total + i
total = t2
t3 = i - 1
i = t3
goto L1
L2:
t4 = total >= 10
t5 = passed && t4
ifFalse t5 goto L3
print total
goto L4
L3:
print i
L4:

Compilation Result
------------------
Compilation succeeded without lexical, syntax, or semantic errors.
```

Figure 3. Actual final run showing AST output and generated TAC.

12. Semantic Analysis

The semantic analyzer performs a depth-first walk of the AST. It enters and exits symbol-table scopes with block traversal, resolves identifiers before use, assigns result types to expression nodes, and records errors in the shared semantic context.

Required rule	Detection strategy	Error code
Undeclared use	active and historical lookup both fail	ERR_SEMANTIC_UNDECLARED_VAR
Scope violation	historical symbol exists but is inactive	ERR_SEMANTIC_OUT_OF_SCOPE
Redeclaration	current-scope lookup succeeds before insertion	ERR_SEMANTIC_REDECLARATION
Type mismatch	initializer/result incompatible with declaration	ERR_SEMANTIC_TYPE_MISMATCH
Invalid assignment	target and expression types incompatible	ERR_SEMANTIC_ASSIGNMENT_INCOMPATIBLE
Invalid arithmetic	one or both operands are non-numeric	ERR_SEMANTIC_INVALID_ARITHMETIC
Invalid modulo	one or both operands are not int	ERR_SEMANTIC_INVALID_MODULO
Invalid relational	operands are not numeric/compatible	ERR_SEMANTIC_INVALID_RELATIONAL
Invalid logical	operand is not bool	ERR_SEMANTIC_INVALID_LOGICAL
Invalid condition	if/while condition is not bool	ERR_SEMANTIC_CONDITION_NOT_BOOL

12.1 Type Propagation

Literal nodes begin with known types. Identifier types are obtained from active symbol entries. Arithmetic combines numeric types, producing float when safe widening is required; relational and equality operations produce bool; logical operations require and produce bool. Assignment and declaration nodes retain their target type for later printing and code generation.

12.2 Cascade Control

When a child expression already has TYPE_ERROR, parent checks propagate that result without issuing derivative messages. This keeps diagnostics focused: one root problem does not become several misleading follow-on errors.

12.3 Warnings and Optimization

Warnings are structurally separate from errors. The infrastructure supports unused-variable, implicit-conversion, style, and dead-code warning categories. Optional constant folding evaluates safe literal-only expressions after mandatory semantic validation. Neither feature changes the fixed grammar.

Figure 4. Actual semantic type-mismatch diagnostic.

12.4 Actual Diagnostic Walkthrough

The captured program assigns an integer expression to a variable declared as bool. The analyzer resolves the target declaration first, evaluates the right-hand side as int, and compares the two types before assignment. It reports the expected and actual types, the source line, and ERR_SEMANTIC_TYPE_MISMATCH. The process then returns a non-zero status and does not generate TAC for the invalid tree.

This behavior is important for grading because the diagnostic demonstrates interaction among the AST, symbol table, and semantic phase rather than a parser-only rejection. The source is syntactically valid; only type-aware analysis can detect the error.

13. Symbol Table

13.1 Stored Information

Field	Required?	Purpose
name	Yes	identifier spelling
type	Yes	int, float, or bool
scope_level	Yes	lexical block containing declaration
declared_line	Yes	diagnostic source line
active	Additional	visibility after block exit
usage_count / lines	Additional	cross-reference and analysis support

13.2 Scope Strategy

The table maintains a current-scope level. Entering a block increments the level. Exiting a block marks symbols declared at that level inactive before returning to the parent scope. Active lookup selects the nearest visible declaration. Current-scope lookup is used for redeclaration checks, and historical lookup distinguishes an undeclared name from an out-of-scope name.

This approach supports nested blocks without deleting records needed for diagnostics and final reporting. It also makes the complete symbol table printable after analysis.

13.3 Scope Example

```
int x;
if (true) {
    int y;
    print x;   // global x is visible
}
print y;       // scope violation: y is inactive
```

The global declaration x remains active at scope 0. The declaration y is inserted at scope 1 and marked inactive when the block exits; historical lookup therefore classifies the final use as an out-of-scope error rather than an undeclared error.

13.4 Demonstration Output

```
=== Detailed Symbol Table (Scope Level: 0) ===
All Symbols:
  - done : bool (declared at line 3, scope 0) [ACTIVE]
  - y : int (declared at line 2, scope 0) [ACTIVE]
  - x : int (declared at line 1, scope 0) [ACTIVE]
```

14. Intermediate Code

14.1 Representation

TAC is stored as a dynamic sequence of instruction strings. The generator maintains independent counters for temporary variables (t1, t2, ...) and labels (L1, L2, ...). Expression generation returns the name containing each result, allowing parent expressions to compose naturally.

Construct	Generated TAC pattern
Assignment	x = value
Binary expression	t1 = left op right
Unary expression	t1 = op operand
Print	print value
Conditional branch	ifFalse condition goto Lx
Unconditional branch	goto Lx
Label	Lx:

14.2 Control-Flow Translation

An if statement creates a false label. An if-else statement also creates an end label and emits an unconditional jump after the true branch. A while statement creates a loop-start label, a false exit, and a backward jump. Conditions are first generated as ordinary expression temporaries, making relational and logical handling reusable.

14.3 Representative Output

```
x = 10
y = 0
done = false
L1:
t1 = x > 0
ifFalse t1 goto L2
t2 = y + x
y = t2
t3 = x - 1
x = t3
goto L1
L2:
t4 = y > 50
ifFalse t4 goto L4
done = true
L4:
```

14.4 TAC Validation

Before output is accepted, tac_validate checks label definitions and references for consistency. Validation state is reset for each program so repeated use cannot preserve stale errors. This extra pass catches generator inconsistencies before a successful compilation result is printed.

15. Challenges and Resolutions

Challenge	Resolution
Correct expression precedence	layered CFG and one parser level per precedence band
Malformed input causing no progress	panic-mode synchronization plus explicit progress guard
Lexical failures counted as syntax failures	independent phase counters and summaries
Nested-scope diagnostics	inactive historical symbols distinguish scope violation from undeclared use
Cascading semantic errors	TYPE_ERROR propagation suppresses derivative diagnostics
Build portability with Flex/Bison requirement	formal .l/.y files plus reviewed portable C11 default build
Memory retained on failure paths	lexer, AST, symbol, semantic, and TAC destructors verified by sanitizers
TAC label consistency	post-generation validator checks referenced and defined labels

15.1 Build and Warning Cleanup

The final build was normalized around object files and dependency generation. Missing standard headers, incorrect function calls, enum mismatches, formatting issues, and stale generated-target interactions were corrected. GNU Make built-in suffix rules were disabled so the formal .l file cannot accidentally overwrite the reviewed scanner source.

15.2 Test Correction

Several test files contained constructs outside the fixed language, including C-style casts and accidental same-scope redeclarations in valid cases. They were rewritten using only the instructor-defined language, and missing lexical/syntax edge cases were added. The resulting corpus now accurately tests the submission rather than testing unsupported C syntax.

16. Testing

16.1 Test Strategy

Valid programs must complete all phases and return exit code 0. Invalid programs must produce an expected lexical, syntax, or semantic diagnostic and return a non-zero exit code. The Python runner discovers every file under tests/valid and tests/invalid, executes the compiler, and verifies expected success or failure behavior.

Expected output snapshots are stored under tests/expected/valid and tests/expected/invalid for grading reference. The make expected target regenerates them only when intended output changes have been reviewed.

16.2 Final Results

Verification	Coverage	Result
Valid end-to-end programs	13 programs	13/13 passed
Invalid programs	16 lexical, syntax, scope, and type cases	16/16 passed
Complete regression suite	29 programs	29/29 passed
Strict compiler build	all C sources	passed; no warnings
Address/UB sanitizers	all 29 tests	passed
TAC validation	every valid generated program	passed

16.3 Coverage Summary

Test family	Representative files
Core success	declaration, assignment, arithmetic, complete_program
Control flow	if_else, while, nested_blocks, deep_nesting
Types and expressions	float_arithmetic, logical_expressions, complex_expressions
Lexical failure	lexical_error, invalid_logical_token, unterminated_comment, unterminated_string
Syntax failure	missing_identifier, missing_semicolon, syntax_error
Semantic failure	undeclared_variable, redeclaration, scope_violation, type_mismatch, invalid_assignment, logical/modulo/condition errors

16.4 Reproducible Commands

```
make clean && make
make test
./compiler tests/valid/complete_program.md
./compiler tests/invalid/lexical_error.md
./compiler tests/invalid/type_mismatch.md
make sanitize FILE=tests/valid/complete_program.md
```

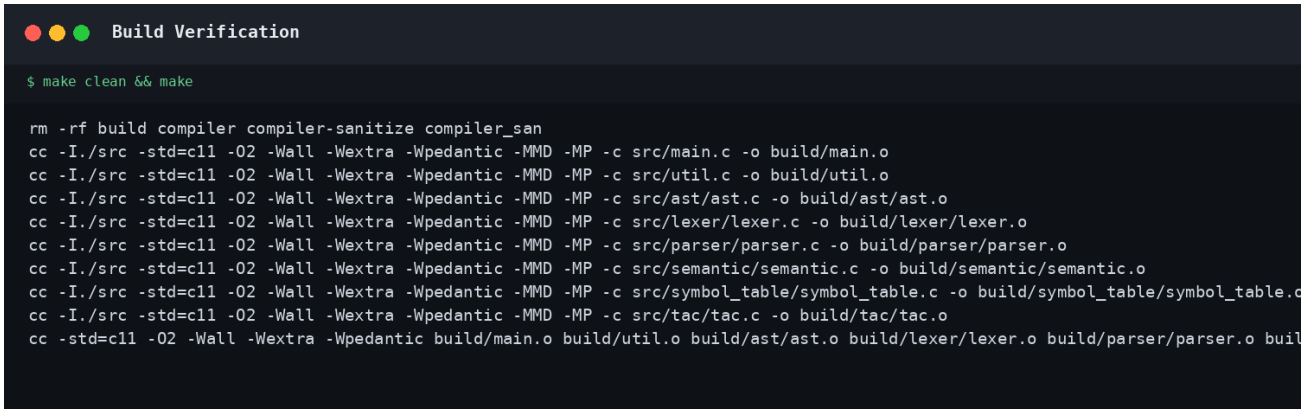

Figure 5. Clean build and complete regression-suite result.

16.5 Verification Interpretation

The clean build proves that all phase interfaces agree under strict C11 compilation. The regression summary then confirms both sides of compiler behavior: every valid case reaches the success path, while every invalid case is rejected. This prevents a misleading implementation that accepts everything or fails indiscriminately.

The sanitizer sweep adds a separate quality check that ordinary functional tests cannot provide. It exercises success and failure cleanup paths and verifies that no address, leak, or undefined-behavior report is produced for the corpus. Together, these checks provide reproducible evidence suitable for final submission and live demonstration.

17. Conclusion

The final project satisfies the complete required pipeline: lexical analysis, syntax analysis, AST construction, nested-scope symbol management, semantic validation, and TAC generation. The components are not isolated demonstrations; they exchange shared tokens, AST nodes, types, symbols, and generated instructions in one executable workflow.

The project demonstrates that compiler correctness depends equally on accepting valid programs and rejecting invalid programs accurately. Clear phase-specific diagnostics, non-zero failure statuses, panic-mode parser recovery, and deterministic cleanup make the implementation suitable for live demonstration and technical questioning. The modular directory structure and formal Flex/Bison sources also make the design easy to inspect during evaluation.

The most important learning outcome was understanding how information becomes progressively richer through the compiler: characters become tokens, tokens become structure, structure receives type and scope meaning, and validated meaning becomes linear intermediate code.

18. References

1. Metropolitan University, Bangladesh. Compiler Construction Lab Project Manual. 2026.
2. Aho, A. V.; Lam, M. S.; Sethi, R.; Ullman, J. D. Compilers: Principles, Techniques, and Tools, 2nd edition. Pearson.
3. Paxson, V. et al. Flex: The Fast Lexical Analyzer - User Manual.
4. Free Software Foundation. GNU Bison Manual.
5. GNU Project. GCC documentation and C language compilation options.
6. Compiler Construction Laboratory course lectures, grammar exercises, and lab examples.

Reference Use in This Project

The course manual is the authoritative source for the required language, phases, report structure, tests, and deliverables. Compiler textbooks and tool manuals were consulted for standard terminology, lexical patterns, grammar design, semantic traversal, and intermediate-code conventions. The submitted team remains responsible for understanding and explaining the implementation.

Appendix A. Repository Map

```
project-root/
|- docs/                report, specification, diagrams, screenshots
|- examples/            representative source programs
|- scripts/             automated regression test runner
|- src/
|  |- lexer/            scanner and Flex specification
|  |- parser/           parser and Bison grammar
|  |- ast/              AST nodes and visualization
|  |- semantic/         type and semantic rules
|  |- symbol_table/     declaration and scope management
|  `- tac/              intermediate-code generation
|- tests/
|  |- valid/
|  |- invalid/
|  `- expected/
|- Makefile
`- README.md
```

Appendix B. Demonstration Checklist

1. Run make clean && make to demonstrate a clean warning-free build.
2. Run make test and show the 29/29 result.
3. Compile tests/valid/complete_program.md and identify its AST, symbols, and TAC.
4. Compile tests/invalid/lexical_error.md to show a lexical diagnostic.
5. Compile tests/invalid/syntax_error.md to show grammar diagnostics and recovery.
6. Compile tests/invalid/type_mismatch.md and scope_violation.md to show semantic checks.
7. Explain the path from source input to AST, symbol table, and TAC during the viva.

Appendix C. Final Submission Checklist

Item	Status
All required modules implemented	Complete
Project builds with make	Verified
Valid and invalid tests included	Complete
Expected outputs included	Complete
Screenshots included	Complete
Project report in PDF format	Complete
README build/run commands	Complete
Formal CFG documented	Complete
Flex and Bison source specifications included	Complete
Generated binaries excluded from final ZIP	Complete